



University of Asia Pacific
Department of Computer Science and Engineering
CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 04

Experiment Title: Design and Simulation of an 8-Bit Register Using NAND Gates and Flip-Flops

Submitted by:

Name: Rifah Tasfia

Student ID: 22201126

Section: C1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 01/10/2026

1. Experiment Name

Designing and Simulating an 8-Bit Register with NAND Gates and Flip-Flops

2. Objective

In this experiment, we want to:

1. Design an 8-bit register using structural VHDL.
2. Build the register from flip-flop modules.
3. Make the flip-flop logic from NAND gates.
4. Learn hierarchical and structural design in VHDL.
5. Create and study the RTL schematic of the 8-bit register in Xilinx ISE.
6. Simulate the register with a testbench and check that it works.
7. Learn how single flip-flops can be joined to make a storage unit with many bits.

3. Apparatus / Hardware & Software Requirements

- List all required tools and components:

- Xilinx ISE 14.7
- VHDL
- ISim Simulator
- A computer for design and simulation

4. Design Method

We designed the 8-bit register in a hierarchical, structural way. We did not describe the whole 8-bit register in one piece using behavioral VHDL. Instead, we split the design into smaller modules. The order of the modules is:

NAND Gate → Flip-Flop → 4-Bit Register → 8-Bit Register

The full 8-bit register has two 4-bit register parts:

- **REG_LOW:** stores the lower 4 bits
- **REG_HIGH:** stores the upper 4 bits

Each 4-bit register has four flip-flops, and each flip-flop is made from NAND gates. So the whole structure looks like this:

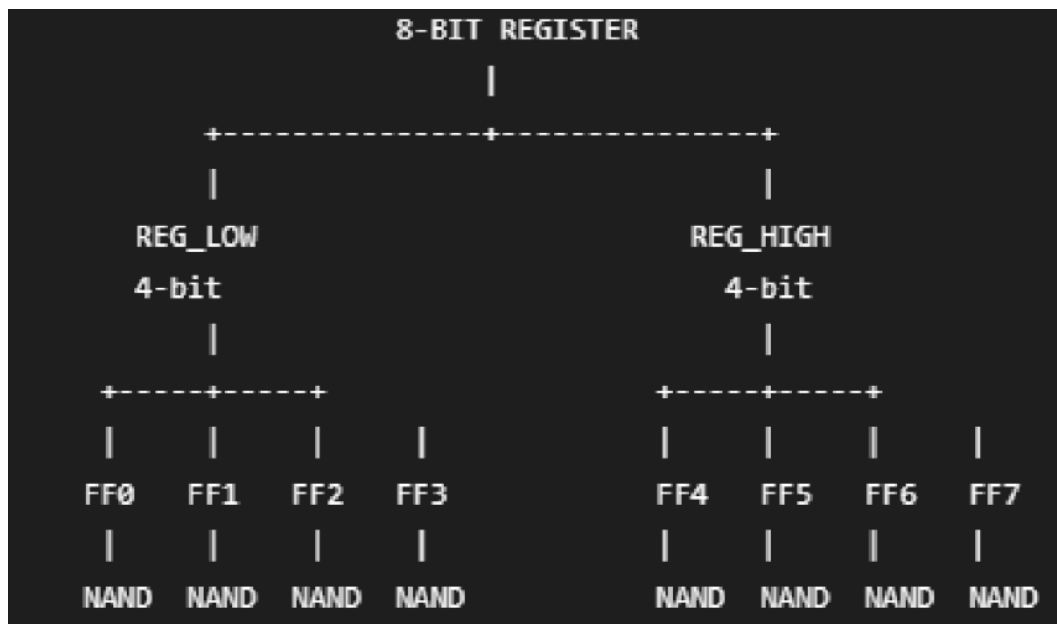


Figure 1: Overall hierarchical structure of the 8-bit register

The ISE hierarchy screenshot should show these items:

- register
- register_8bit
- REG_LOW
- REG_HIGH
- the flip-flops
- NAND gates.

5. RTL Schematic Diagrams

5.1 Top-Level RTL Schematic

The top-level RTL schematic shows the whole 8-bit register. It also shows how the lower and upper 4-bit registers are connected. The 8-bit register is split into REG_LOW and REG_HIGH.

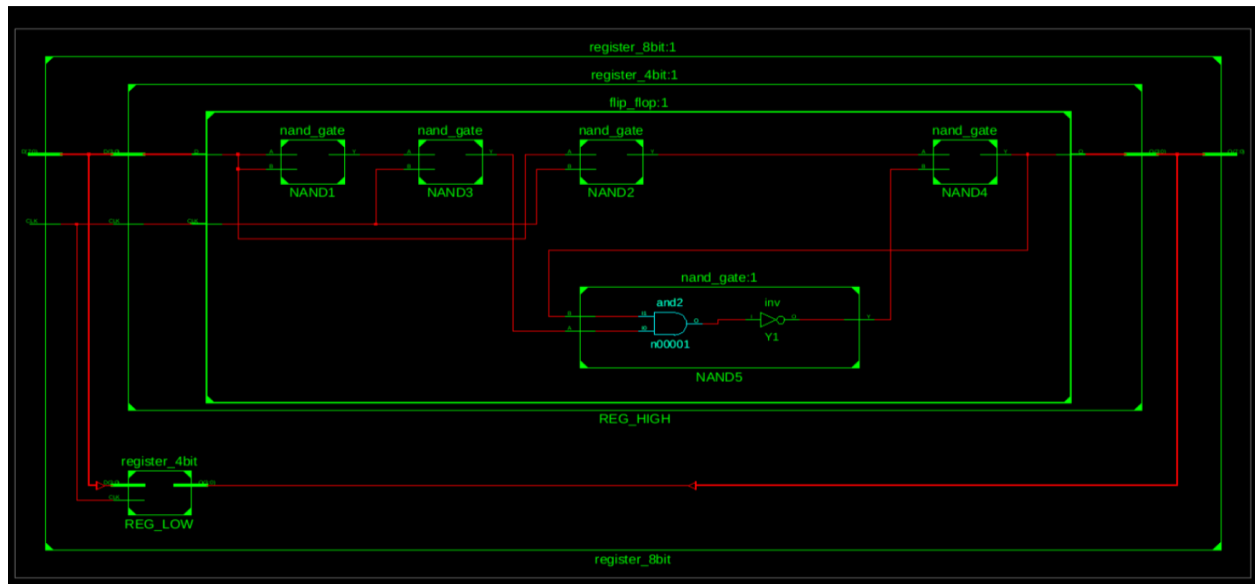


Figure 2: RTL schematic of the complete 8-bit register

5.2 4-Bit Register RTL Schematic

Each 4-bit register is made from four flip-flops. REG_LOW and REG_HIGH use the same structural design.

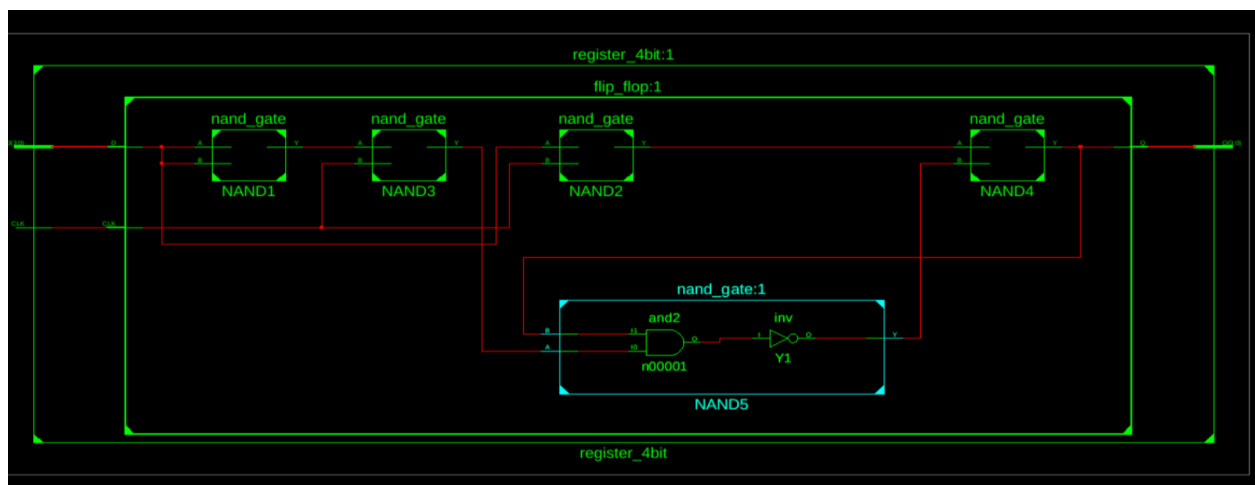


Figure 3: RTL schematic of the 4-bit register

6. Flip-Flop Design with NAND Gates

We built each flip-flop from NAND gates. We did this on purpose, to show how storage (sequential) logic can be made from simple gates, instead of using a ready-made register part.

The structure of one flip-flop is shown below.

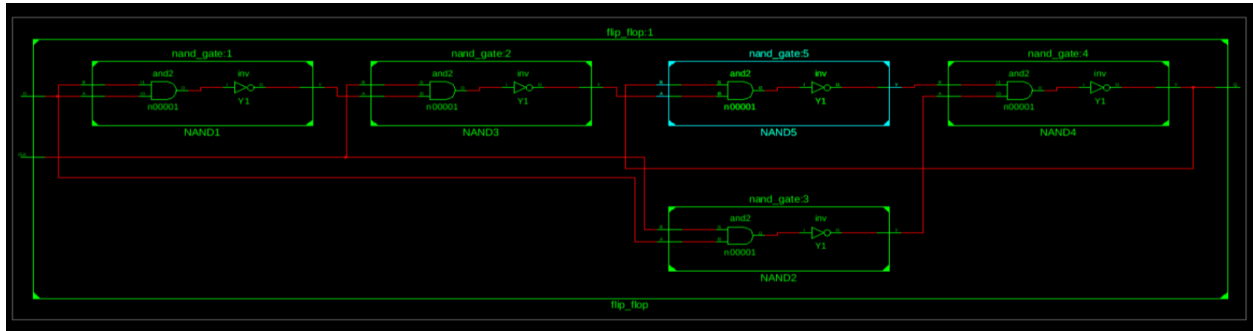


Figure 4: Inside view of a flip-flop made of NAND gates

The NAND gates give the basic logic needed to hold a binary value. In the Xilinx ISE hierarchy, we can open a flip-flop module to see the NAND gates inside it.

7. NAND Gate Design

A NAND gate is a universal logic gate. A two-input NAND gate gives the opposite result of an AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NAND gate was implemented using VHDL Dataflow modeling.

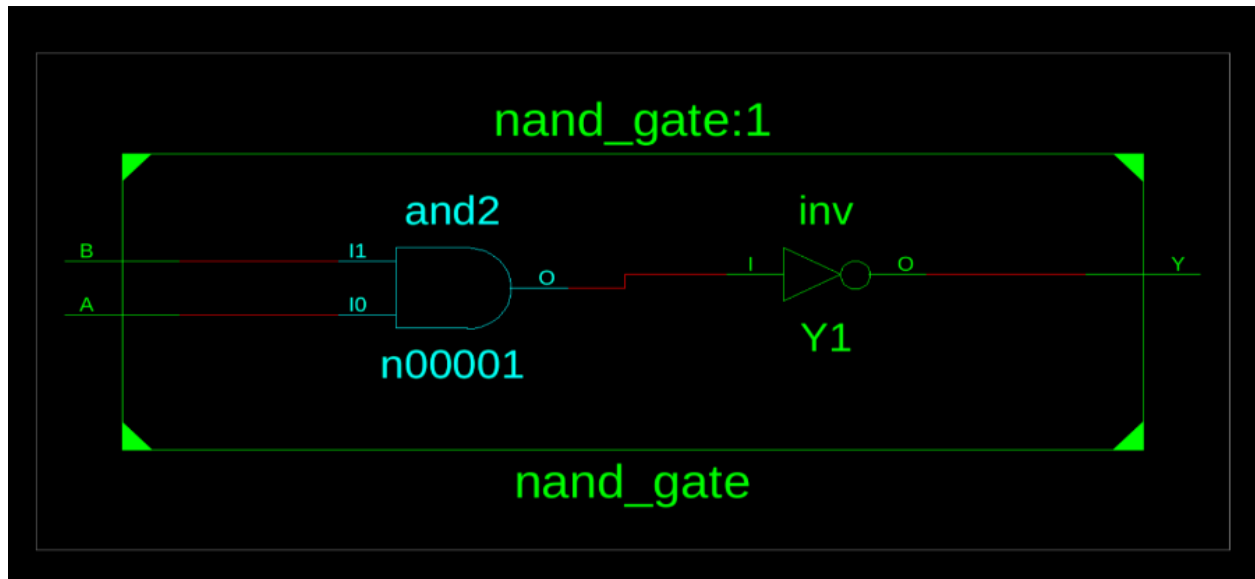


Figure 5: RTL schematic of the NAND gate.

8. Structural VHDL Design

We wrote the full design in structural VHDL. It has separate VHDL modules for:

1. NAND gate
2. Flip-flop
3. 4-bit register
4. 8-bit register
5. Testbench

The modules are used inside each other, level by level. The file order is:

```
nand_gate.vhd
↓
flip_flop.vhd
↓
register_4bit.vhd
↓
register_8bit.vhd
↓
register_8bit_tb.vhd
```

This modular way makes the design easier to understand, simulate, test, and change.

9. GitHub Repository

All VHDL source code, testbenches, and other project files are kept in a GitHub repository:

<https://github.com/Rifah03/VLSI-ISE-14.7>

10. Simulation

We simulated the 8-bit register in Xilinx ISE (ISim) with a VHDL testbench. The testbench sends input signals to the register and watches the output. The simulation checks that:

- The register accepts 8-bit input data.
- The data is stored correctly.
- The outputs match the stored data.
- The flip-flops work correctly.
- The whole hierarchical design behaves as expected.

10.1 Simulation Waveform

The waveform shows different inputs being applied to the register. For example:

- Input: 10101010
- The register stores the input.
- Output: 10101010

After that, another input is applied to check that the register updates correctly under the clock/control operation of the design.

The waveform shows the clock, the input bits, and the output bits. It also shows any reset or control signal used in the design.

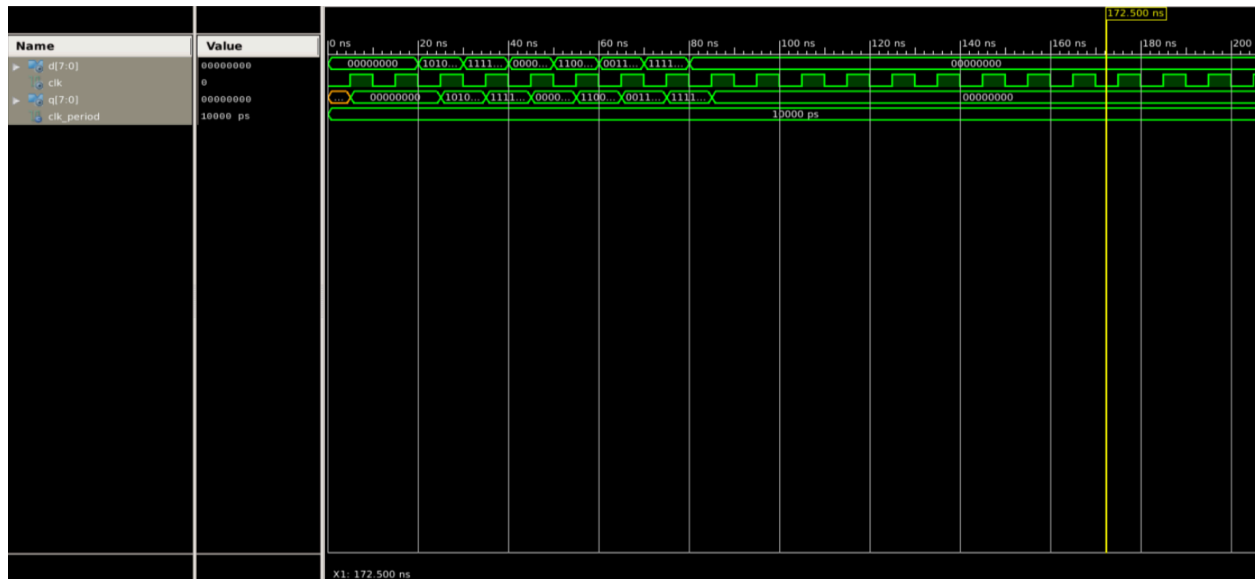


Figure 6: ISim simulation waveform of the 8-bit register

11. Simulation Analysis

To study the waveform, we compare the input and output signals at the right clock or control changes. When valid input data is applied and the needed clock/control condition happens, the flip-flops store the matching binary values.

For example, if the input is $D[7:0] = 10110101$, the register should store it and give the output $Q[7:0] = 10110101$.

So the simulation confirms how the complete 8-bit register stores data.

12. Conclusion

In this experiment, we designed and simulated an 8-bit register using structural VHDL in Xilinx ISE. We built the register step by step from smaller digital parts.

The basic building block was the NAND gate, and we used it to make the flip-flop. We joined several flip-flops to make 4-bit registers, and we joined two 4-bit registers to make the complete 8-bit register.

The RTL schematic and the Xilinx ISE hierarchy showed the inside structure of the design. The simulation waveform was used to check that the register works correctly.

This experiment gave us practical understanding of hierarchical structural VHDL design, NAND-based sequential logic, building flip-flops, how registers are organized, RTL synthesis, and digital circuit simulation.